Print copy of official Microsoft Learn documentation

Source: https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/overview
Exported: 2026-10-14. Source metadata date: 2025-01-19T00:00:00Z.

Microsoft and documentation contributors. Article wording and code for the requested version are retained; web navigation is removed and print formatting is added. This locally generated PDF is not a Microsoft-published PDF book or a complete product manual. Source-page attribution and notices apply.

Documentation license and terms: Microsoft Learn terms; .NET documentation: CC BY 4.0 license. Check the source for specific attribution or exceptions.

JSON serialization and deserialization in .NET - overview

The System.Text.Json namespace provides functionality for serializing to and deserializing from (or marshalling and unmarshalling) JavaScript Object Notation (JSON). *Serialization* is the process of converting the state of an object, that is, the values of its properties, into a form that can be stored or transmitted. The serialized form doesn't include any information about an object's associated methods. *Deserialization* reconstructs an object from the serialized form.

The `System.Text.Json` library design emphasizes high performance and low memory allocation over an extensive feature set. Built-in UTF-8 support optimizes the process of reading and writing JSON text encoded as UTF-8, which is the most prevalent encoding for data on the web and files on disk.

The library also provides classes for working with an in-memory document object model (DOM). This feature enables random access to the elements in a JSON file or string.

For Visual Basic, there are some limitations on what parts of the library you can use. For more information, see Visual Basic support.

How to get the library

The library is built-in as part of the shared framework for .NET Core 3.0 and later versions. The source generation feature is built-in as part of the shared framework for .NET 6 and later versions.

For framework versions earlier than .NET Core 3.0, install the System.Text.Json NuGet package. The package supports:

- .NET Standard 2.0 and later
- .NET Framework 4.6.2 and later
- .NET 8 and later

Namespaces and APIs

- The System.Text.Json namespace contains all the entry points and the main types.

- The System.Text.Json.Serialization namespace contains attributes and APIs for advanced scenarios and customization specific to serialization and deserialization.
- The System.Net.Http.Json namespace contains extension methods for serializing and deserializing JSON payloads from the network.

> Important
>
> `System.Text.Json` doesn't support the following serialization APIs that you might have used previously:
>
> - Attributes from the System.Runtime.Serialization namespace.
> - The System.SerializableAttribute attribute and the ISerializable interface. These types are used only for Binary and XML serialization.

Reflection vs. source generation

By default, `System.Text.Json` gathers the metadata it needs to access properties of objects for serialization and deserialization *at runtime* using reflection. As an alternative, `System.Text.Json` can use the C# source generation feature to improve performance, reduce private memory usage, and facilitate assembly trimming, which reduces app size.

For more information, see Reflection versus source generation.

Security information

For information about security threats that were considered when designing JsonSerializer, and how they can be mitigated, see `System.Text.Json` Threat Model.

Thread safety

The `System.Text.Json` serializer was designed with thread safety in mind. Practically, this means that once locked, JsonSerializerOptions instances can be safely shared across multiple threads. JsonDocument provides an immutable, and in .NET 8 and later versions, thread-safe, DOM representation for JSON values.

Additional resources

- How to use the library